

Практическая работа 5: "Классы и объекты в Kotlin"

Тема

Классы и объекты в Kotlin. Основы объектно-ориентированного программирования.

Цель

Научиться создавать и использовать классы и объекты в Kotlin, понимать основные принципы ООП (инкапсуляция, наследование, полиморфизм).

Ход работы

1. Изучить теоретический материал по классам и объектам.
2. Выполнить практический пример.
3. Решить задание согласно вашему варианту.
4. Ответить на контрольные вопросы.

Теоретическая часть

Классы в Kotlin

Класс — это шаблон для создания объектов. Он определяет свойства (данные) и методы (функции), которые будут у объектов.

Пример объявления класса:

```
class SmartDevice {  
    val name = "Android TV"  
    var deviceStatus = "online"  
  
    fun turnOn() {  
        println("Device is turned on")  
    }  
  
    fun turnOff() {  
        println("Device is turned off")  
    }  
}
```

Объекты и экземпляры классов

Объект — это экземпляр класса. Мы можем создавать multiple объектов одного класса.

Пример создания объекта:

```
val smartTv = SmartDevice()
smartTv.turnOn()
```

Конструкторы

Конструкторы используются для инициализации объектов при их создании.

Пример конструктора:

```
class SmartDevice(val name: String, val category: String) {
    var deviceStatus = "online"

    // методы класса
}
```

Наследование

Наследование позволяет создавать новый класс на основе существующего.

Пример наследования:

```
open class SmartDevice(val name: String, val category: String)

class SmartTvDevice(name: String, category: String) :
    SmartDevice(name, category) {
    // дополнительные свойства и методы
}
```

Практический пример

Задача: Создать систему управления умным домом с устройствами разных типов.

```
// Базовый класс для всех умных устройств
open class SmartDevice(val name: String, val category: String) {
    var deviceStatus = "online"
    protected set

    open fun turnOn() {
        deviceStatus = "on"
        println("$name is turned on")
    }

    open fun turnOff() {
        deviceStatus = "off"
        println("$name is turned off")
    }
}
```

```
}

// Класс для умного телевизора
class SmartTvDevice(name: String, category: String) :
    SmartDevice(name, category) {

    private var volume = 5
    private var channel = 1

    fun increaseVolume() {
        if (volume < 10) volume++
        println("Volume increased to $volume")
    }

    fun changeChannel(newChannel: Int) {
        channel = newChannel
        println("Channel changed to $channel")
    }

    override fun turnOn() {
        super.turnOn()
        println("TV is ready with volume $volume and channel $channel")
    }
}

// Класс для умной лампы
class SmartLightDevice(name: String, category: String) :
    SmartDevice(name, category) {

    private var brightness = 50

    fun increaseBrightness() {
        if (brightness < 100) brightness += 10
        println("Brightness increased to $brightness")
    }

    override fun turnOn() {
        super.turnOn()
        println("Light is on with brightness $brightness")
    }
}

// Класс для управления умным домом
class SmartHome {
    private val devices = mutableListOf<SmartDevice>()

    fun addDevice(device: SmartDevice) {
        devices.add(device)
        println("Device ${device.name} added to smart home")
    }

    fun turnOnAllDevices() {
        devices.forEach { it.turnOn() }
    }
}
```

```
fun turnOffAllDevices() {
    devices.forEach { it.turnOff() }
}

fun getDeviceStatus(): String {
    return "Total devices: ${devices.size}. " +
        "Online: ${devices.count { it.deviceStatus == "on" }}"
}

}

// Использование системы
fun main() {
    val smartHome = SmartHome()

    val tv = SmartTvDevice("Living Room TV", "Entertainment")
    val light = SmartLightDevice("Kitchen Light", "Lighting")

    smartHome.addDevice(tv)
    smartHome.addDevice(light)

    smartHome.turnOnAllDevices()
    tv.increaseVolume()
    tv.changeChannel(5)
    light.increaseBrightness()

    println(smartHome.getDeviceStatus())

    smartHome.turnOffAllDevices()
}
```

Варианты заданий

Вариант 1: Создать класс "Банковский счет" с методами пополнения, снятия и проверки баланса.

Вариант 2: Создать класс "Студент" с информацией о студенте и методами для работы с оценками.

Вариант 3: Создать класс "Книга" с информацией о книге и методами для работы с состоянием (взята/возвращена).

Вариант 4: Создать класс "Автомобиль" с методами запуска двигателя, остановки и изменения скорости.

Вариант 5: Создать класс "Магазин" с товарами и методами добавления, удаления и поиска товаров.

Вариант 6: Создать класс "Библиотека" с книгами и методами выдачи и приема книг.

Вариант 7: Создать класс "Игрок" с характеристиками и методами для игры (атака, защита).

Вариант 8: Создать класс "Прямоугольник" с методами вычисления площади и периметра.

Вариант 9: Создать класс "Калькулятор" с базовыми математическими операциями.

Вариант 10: Создать класс "Задача" с информацией о задаче и методами изменения статуса.

Вариант 11: Создать класс "Пользователь" с методами регистрации, авторизации и изменения данных.

Вариант 12: Создать класс "Товар" с информацией о товаре и методами применения скидки.

Вариант 13: Создать класс "Животное" с характеристиками и методами кормления, игры.

Вариант 14: Создать класс "Фильм" с информацией о фильме и методами аренды, возврата.

Вариант 15: Создать класс "Погода" с данными о температуре и методами прогнозирования.

Критерии оценки

5 (отлично):

- Полностью реализована требуемая функциональность
- Код хорошо структурирован и соответствует принципам ООП
- Присутствуют комментарии к коду
- Программа работает без ошибок

4 (хорошо):

- Реализована основная функциональность
- Код в целом соответствует принципам ООП
- Возможны незначительные недочеты в структуре кода

3 (удовлетворительно):

- Реализована базовая функциональность
- Есть нарушения принципов ООП
- Код требует улучшения структуры

2 (неудовлетворительно):

- Функциональность не реализована или реализована некорректно
- Серьезные нарушения принципов ООП
- Код не работает или работает с ошибками

Контрольные вопросы

1. Что такое класс и объект в Kotlin?
2. Чем отличается класс от объекта?
3. Что такое конструктор класса?
4. Какие модификаторы доступа существуют в Kotlin?
5. Что такое наследование и как оно реализуется в Kotlin?
6. Что такое инкапсуляция и как она достигается в классах?
7. Чем отличается открытый (open) класс от обычного?
8. Как создать объект класса в Kotlin?
9. Что такое свойства класса и чем они отличаются от переменных?
10. Как переопределить метод родительского класса?

Структура репозитория

```
praktika-klassy-obekty/  
├── src/  
│   ├── main/  
│   │   └── kotlin/  
│   │       └── Main.kt  
├── images/  
│   ├── screenshot1.png  
│   └── screenshot2.png  
├── .gitignore  
└── README.md
```

README.md шаблон

```
# Практическая работа №X: Классы и объекты в Kotlin  
  
**Вариант:** [номер вашего варианта]  
  
**Задание:** [текст задания согласно вашему варианту]  
  
## Код программы  
  
## Скриншоты работы программы  
  
![Результат работы программы](images/screenshot1.png)  
![Результат работы программы](images/screenshot2.png)
```